

DOCUMENTAÇÃO DO PROJETO

SERVIDOR WEB COM NODE.JS

Júlia Lins Pereira da Silva

Vitor Antonio Romualdo

Desenvolvimento de Software Multiplataforma — DSM — 3º Semestre

FATEC Luigi Papaiz — Diadema

Desenvolvimento Web III

Professor Vinicius Heltai

SUMÁRIO

1. Objetivo do projeto	3
2. Estrutura das rotas desenvolvidas	3
3. Explicação do funcionamento da aplicação	5
4. Códigos-fonte utilizados no projeto	7
5. Explicação dos principais trechos de código	10
5.1 Criação do servidor	10
5.2 Identificação das rotas	10
5.3 Código de status HTTP	11
5.4 Diferenciação entre HTML e PDF	12
6. Justificativa das decisões adotadas	13
7. Considerações finais	14

1. Objetivo do projeto

O projeto tem como objetivo desenvolver um servidor web utilizando **Node.js**, aplicando os conceitos estudados na disciplina de Desenvolvimento Web III, principalmente a criação de um servidor HTTP, utilização de sistemas de rotas e manipulação de arquivos por meio do módulo File System (fs).

Além da implementação técnica do servidor, o projeto apresenta informações pessoais e profissionais dos integrantes da dupla, disponibilizando páginas distintas para apresentação pessoal e acesso aos respectivos currículos em formato PDF.

Dessa forma, o projeto busca demonstrar na prática a utilização de rotas para organizar diferentes conteúdos dentro de uma mesma aplicação web.

2. Estrutura das rotas desenvolvidas

A aplicação foi organizada utilizando diferentes rotas para cada parte do projeto.

ROTA	FUNÇÃO
/	Página principal do projeto
/projeto	Acesso à documentação do projeto em PDF
/julia	Página inicial da apresentação da Júlia
/julia/sobre	Apresentação pessoal e competências da Júlia
/julia/curriculo	Acesso ao currículo da Júlia em PDF

/vitor	Página inicial da apresentação do Vitor
/vitor/sobre	Apresentação pessoal e competências do Vitor
/vitor/curriculo	Acesso ao currículo do Vitor em PDF

A estrutura física dos arquivos foi organizada da seguinte maneira:

```

  ✓ Projeto-NodeJS-DWIII
    ✓ julia
      ✓ curriculo
        📄 curriculo.pdf
      ✓ sobre
        <> sobre.html
        <> index.html
    ✓ projeto
      📄 documentacao.pdf
    ✓ vitor
      ✓ curriculo
        <> curriculopdf
      ✓ sobre
        <> sobre.html
        <> index.html
    JS app.js
    <> index.html
```

Essa organização permite separar os arquivos de cada integrante e facilita a manutenção do projeto.

3. Explicação do funcionamento da aplicação

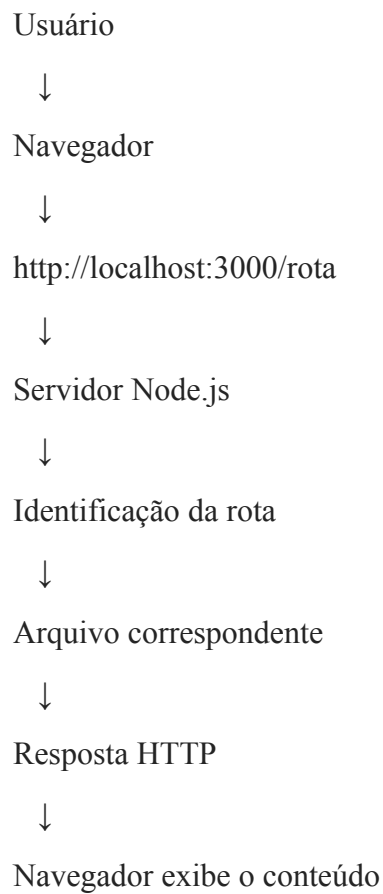
A aplicação utiliza o módulo nativo `http` do Node.js para criar um servidor web.

Quando o usuário acessa:

```
http://localhost:3000/
```

O servidor verifica a URL solicitada e identifica qual conteúdo deve ser enviado ao navegador.

O funcionamento pode ser representado da seguinte forma:



Por exemplo, quando o usuário acessa:

`/julia/sobre`

o servidor identifica essa rota:

```
else if(parts.path == "/julia/sobre"){  
    response.writeHead(200, {"Content-type": "text/html;  
    charset=utf-8"});  
    readFile(response, "julia/sobre/sobre.html");  
}
```

O Node.js então localiza o arquivo:

`julia/sobre/sobre.html`

e envia seu conteúdo para o navegador.

No caso dos arquivos PDF, o processo é semelhante, porém o tipo de conteúdo informado na resposta HTTP é diferente.

Por exemplo:

```
else if(parts.path == "/julia/curriculo"){  
    response.writeHead(200, {"Content-type":  
    "application/pdf"});  
    readFile(response, "julia/curriculo/curriculo.pdf");  
}
```

Nesse caso, o navegador recebe o arquivo como um documento PDF.

4. Códigos-fonte utilizados no projeto

O código utilizado para processar as rotas e o servidor web:

```
// Carregar módulos

const http = require("http");

const url = require("url");

const fs = require("fs");


// Módulo que lê arquivos

function readFile(response, file){

    fs.readFile(file, function(err, data){

        response.end(data);

    });

}


// Função Callback

var callback = function(request, response){

    var parts = url.parse(request.url);


    // Rota principal do projeto

    if(parts.path == "/"){

        response.writeHead(200, {"Content-type": "text/html; charset=utf-8"});

        readFile(response, "index.html");

    }


    // Rotas de apresentação: Júlia
```

```
    else if(parts.path == "/julia"){
        response.writeHead(200, {"Content-type": "text/html; charset=utf-8"});
        readFile(response, "julia/index.html");
    }
    else if(parts.path == "/julia/sobre"){
        response.writeHead(200, {"Content-type": "text/html; charset=utf-8"});
        readFile(response, "julia/sobre/sobre.html");
    }
    else if(parts.path == "/julia/curriculo"){
        response.writeHead(200, {"Content-type": "application/pdf"});
        readFile(response, "julia/curriculo/curriculo.pdf");
    }

    // Rotas de apresentação: Vitor
    else if(parts.path == "/vitor"){
        response.writeHead(200, {"Content-type": "text/html; charset=utf-8"});
        readFile(response, "vitor/index.html");
    }

    else if(parts.path == "/vitor/sobre"){
        response.writeHead(200, {"Content-type": "text/html; charset=utf-8"});
        readFile(response, "vitor/sobre/sobre.html");
    }

    else if(parts.path == "/vitor/curriculo"){
        response.writeHead(200, {"Content-type": "text/html; charset=utf-8"});
```



```
        readFile(response, "vitor/curriculo/curriculo.pdf");
    }

    // Rota de apresentação do projeto
    else if (parts.path == "/projeto"){
        response.writeHead(200, {"Content-type": "application/pdf"})
        readFile(response, "projeto/documentacao.pdf")
    }

    // Caso não tenha sido encontrada nenhuma rota
    else {
        response.writeHead(404, {"Content-type": "text/plain; charset=utf-8"});
        response.end("Página não encontrada!");
    }
};

// Criar e configurar um servidor http
var server = http.createServer(callback);
server.listen(3000);
console.log("Servidor iniciando na porta: http://localhost:3000/");
```

5. Explicação dos principais trechos de código

5.1 Criação do servidor

```
var server = http.createServer(callback);  
  
server.listen(3000);
```

O método `http.createServer()` cria o servidor HTTP e recebe a função `callback`, responsável por processar cada requisição recebida.

Já:

```
server.listen(3000);
```

determina que o servidor ficará disponível na porta `3000`.

Assim, a aplicação pode ser acessada através de:

```
http://localhost:3000/
```

5.2 Identificação das rotas

A aplicação utiliza uma sequência de estruturas condicionais `if` e `else if` para verificar qual rota foi solicitada:

```
if (parts.path == "/") {  
    ...  
}  
  
else if (parts.path == "/julia") {  
    ...  
}  
  
else if (parts.path == "/julia/sobre") {  
    ...  
}
```

A propriedade:

```
parts.path
```

contém o caminho da URL solicitada.

Por exemplo:

```
http://localhost:3000/julia/sobre
```

resulta em:

```
/julia/sobre
```

Assim, o servidor consegue identificar qual página deve retornar.

5.3 Código de status HTTP

Para as páginas existentes, foi utilizado:

```
response.writeHead(200, {  
    "Content-type": "text/html; charset=utf-8"  
});
```

O código 200 indica que a requisição foi processada com sucesso.

Para uma rota inexistente, foi utilizado:

```
response.writeHead(404, {  
    "Content-type": "text/plain; charset=utf-8"  
});
```

O código 404 indica que o recurso solicitado não foi encontrado.

5.4 Diferenciação entre HTML e PDF

Um ponto importante da aplicação é que o servidor não envia todos os arquivos com o mesmo tipo de conteúdo.

Para HTML:

```
"Content-type": "text/html; charset=utf-8"
```

Para PDF:

```
"Content-type": "application/pdf"
```

Essa diferenciação informa ao navegador qual é o tipo do conteúdo recebido.

Por isso, na rota:

```
else if(parts.path == "/projeto"){  
    response.writeHead(200, {"Content-type":  
"application/pdf"})  
    readFile(response, "projeto/documentacao.pdf")  
}
```

O navegador interpreta o arquivo recebido como um documento PDF.

6. Justificativa das decisões adotadas

A organização do projeto em rotas distintas foi escolhida para atender ao objetivo da atividade e demonstrar o funcionamento de um sistema de rotas utilizando Node.js.

Foi criada uma rota principal para cada integrante:

`/julia`

`/vitor`

Essas rotas funcionam como páginas iniciais das apresentações pessoais.

Também foram criadas subrotas específicas:

`/julia/sobre`

`/julia/curriculo`

`/vitor/sobre`

`/vitor/curriculo`

Essa divisão permite separar os diferentes tipos de informação e torna a navegação mais organizada.

A utilização de arquivos HTML separados também foi escolhida para manter cada conteúdo independente. Dessa maneira, alterações na apresentação pessoal, por exemplo, não interferem na página destinada ao currículo.

Para os currículos e para a documentação do projeto, foi utilizado o formato PDF, permitindo disponibilizar documentos completos sem precisar reproduzir seu conteúdo dentro das páginas HTML.

Outra decisão foi utilizar os módulos nativos do Node.js, principalmente `http`, `url` e `fs`, sem a utilização de frameworks externos. Essa escolha permite demonstrar diretamente os conceitos fundamentais trabalhados durante a disciplina, como criação de servidor HTTP, tratamento de requisições, rotas e leitura de arquivos.

7. Considerações finais

O desenvolvimento do projeto possibilitou aplicar na prática os conceitos de criação de servidores web utilizando Node.js e compreender melhor o funcionamento das requisições HTTP e dos sistemas de rotas.

Através da implementação, foi possível compreender como uma URL pode ser utilizada para determinar qual conteúdo deve ser apresentado ao usuário e como o servidor pode acessar diferentes arquivos de acordo com a rota solicitada.

Além da parte técnica, o projeto também possibilitou a criação de uma apresentação pessoal para cada integrante, reunindo informações sobre formação, habilidades, projetos e currículo.

Dessa forma, o projeto cumpriu o objetivo de unir os conhecimentos desenvolvidos em aula com uma aplicação prática, utilizando Node.js, HTTP, sistemas de rotas e o módulo **File System** para construir um servidor web funcional.